
MENU-DRIVEN STRING OPERATIONS IN C

Remove Vowels | First Repeating Alphabet | Anagram Check

Submitted By

Name : Rudra Pratap Singh
SAP ID : 590032811
Batch : 16

1. Problem Statement

Write a menu-driven C program that performs the following operations on strings, with every operation implemented in its own separate function:

- Remove all vowels from a string.
- Find the first repeating lowercase alphabet in a string.
- Check whether two strings are anagrams of each other.

2. Objectives

- To understand how strings are stored and processed as character arrays in C.
- To practise modular programming by writing a separate function for each task.
- To use a menu-driven interface built with a do-while loop and a switch statement.
- To apply frequency-counting arrays and the standard library `<ctype.h>` and `<string.h>`.

3. Concepts Used

Concept	Where it is used
Character arrays	Strings are stored in char arrays terminated by the null character '\0'.
Functions	<code>removeVowels()</code> , <code>firstRepeatingLowercase()</code> , <code>areAnagrams()</code> , <code>isVowel()</code> , <code>readString()</code> .
do-while loop	Keeps the menu running until the user chooses Exit.
switch-case	Selects the operation based on the menu choice.
Frequency array	<code>int count[26]</code> counts how many times each alphabet occurs.
<code>fgets()</code>	Reads a complete line, including spaces, safely within the buffer size.
<code>ctype.h</code> functions	<code>tolower()</code> and <code>isalpha()</code> handle upper/lower case and letters.

4. Algorithms

4.1 Remove All Vowels

1. Start with $i = 0$ (index in input) and $j = 0$ (index in result).
2. Repeat while `str[i]` is not the null character:
3. If `str[i]` is not a vowel (a, e, i, o, u in either case), copy it to `result[j]` and increase j .
4. Increase i .
5. Place the null character at `result[j]` and display result.

4.2 First Repeating Lowercase Alphabet

1. Create an array `count[26]` and set every element to 0.
2. Scan the string once; for every lowercase letter, increase `count[letter - 'a']`.
3. Scan the string a second time from the left.
4. Return the first lowercase letter whose count is greater than 1.
5. If no such letter exists, return the null character and report that nothing repeats.

4.3 Anagram Check

1. Create an array `count[26]` and set every element to 0.
2. For every alphabet in the first string, increase its count (case is ignored, spaces are skipped).
3. For every alphabet in the second string, decrease its count in the same way.
4. If every element of `count[]` is 0, the strings are anagrams; otherwise they are not.

4.4 Main Program (Menu)

1. Display the menu with four options.
2. Read the user's choice. If the input is not a number, show an error and display the menu again.
3. Using switch, call the function for the selected operation and display the result.
4. Repeat until the user selects option 4 (Exit).

5. Source Code

The complete program is given below (file name: string_operations.c).

```
/*
 * Program : Menu-Driven String Operations
 * Author  : Rudra Pratap Singh
 * SAP ID  : 590032811
 * Batch   : 16
 *
 * Operations:
 * 1. Remove all vowels from a string
 * 2. Find the first repeating lowercase alphabet in a string
 * 3. Check whether two strings are anagrams of each other
 */

#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define MAX 200

/* Function prototypes */
void readString(const char prompt[], char str[]);
int  isVowel(char ch);
void removeVowels(const char str[], char result[]);
char firstRepeatingLowercase(const char str[]);
int  areAnagrams(const char s1[], const char s2[]);

int main(void)
{
    int choice;
    char str1[MAX], str2[MAX], result[MAX];
    char ch;

    do
    {
        printf("\n=====\\n");
        printf("          STRING OPERATIONS MENU\\n");
        printf("=====\\n");
        printf(" 1. Remove all vowels from a string\\n");
        printf(" 2. Find first repeating lowercase alphabet\\n");
        printf(" 3. Check if two strings are anagrams\\n");
        printf(" 4. Exit\\n");
        printf("-----\\n");
        printf("Enter your choice (1-4): ");

        if (scanf("%d", &choice) != 1)
        {
            /* Invalid (non-numeric) input: clear the buffer and retry */
            while (getchar() != '\\n')
                ;
            printf("Invalid input! Please enter a number between 1 and 4.\\n");
            choice = 0;
            continue;
        }
        while (getchar() != '\\n') /* discard the leftover newline */
            ;

        switch (choice)
        {
            case 1:
                readString("Enter a string: ", str1);
                removeVowels(str1, result);
                printf("String after removing vowels: \"%s\"\\n", result);
                break;

            case 2:
                readString("Enter a string: ", str1);
                ch = firstRepeatingLowercase(str1);
                if (ch != '\\0')
                    printf("First repeating lowercase alphabet: '%c'\\n", ch);
                else
                    printf("No repeating lowercase alphabet found.\\n");
                break;

            case 3:
                readString("Enter first string : ", str1);
                readString("Enter second string: ", str2);
                if (areAnagrams(str1, str2))
                    printf("Result: The strings ARE anagrams.\\n");
                else
                    printf("Result: The strings are NOT anagrams.\\n");
                break;
        }
    }
}
```

```

        case 4:
            printf("Thank you! Exiting the program.\n");
            break;

        default:
            printf("Invalid choice! Please enter a number between 1 and 4.\n");
    } while (choice != 4);

    return 0;
}

/* Displays a prompt and reads a full line (spaces allowed) into str */
void readString(const char prompt[], char str[])
{
    printf("%s", prompt);
    if (fgets(str, MAX, stdin) == NULL)
    {
        str[0] = '\0';
        return;
    }
    str[strcspn(str, "\n")] = '\0';    /* remove trailing newline */
}

/* Returns 1 if ch is a vowel (upper or lower case), otherwise 0 */
int isVowel(char ch)
{
    ch = tolower((unsigned char)ch);
    return (ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u');
}

/* Copies str into result, skipping every vowel */
void removeVowels(const char str[], char result[])
{
    int i, j = 0;

    for (i = 0; str[i] != '\0'; i++)
    {
        if (!isVowel(str[i]))
        {
            result[j] = str[i];
            j++;
        }
    }
    result[j] = '\0';
}

/*
 * Returns the first lowercase letter (from left to right) that occurs
 * more than once in str. Returns '\0' if there is no such letter.
 */
char firstRepeatingLowercase(const char str[])
{
    int count[26] = {0};
    int i;

    /* Pass 1: count occurrences of every lowercase letter */
    for (i = 0; str[i] != '\0'; i++)
    {
        if (str[i] >= 'a' && str[i] <= 'z')
            count[str[i] - 'a']++;
    }

    /* Pass 2: find the leftmost letter whose count is more than 1 */
    for (i = 0; str[i] != '\0'; i++)
    {
        if (str[i] >= 'a' && str[i] <= 'z' && count[str[i] - 'a'] > 1)
            return str[i];
    }
    return '\0';
}

/*
 * Returns 1 if s1 and s2 are anagrams, otherwise 0.
 * Case and spaces are ignored; only alphabets are compared.
 */
int areAnagrams(const char s1[], const char s2[])
{
    int count[26] = {0};
    int i;

    for (i = 0; s1[i] != '\0'; i++)
        if (isalpha((unsigned char)s1[i]))
            count[tolower((unsigned char)s1[i]) - 'a']++;

```

```
    for (i = 0; s2[i] != '\0'; i++)
        if (isalpha((unsigned char)s2[i]))
            count[tolower((unsigned char)s2[i]) - 'a']--;

    for (i = 0; i < 26; i++)
        if (count[i] != 0)
            return 0;

    return 1;
}
```

6. Function Description

Function	Purpose	Parameters	Returns
readString()	Prints a prompt and reads a full line, then removes the trailing newline.	prompt, str	void
isVowel()	Checks whether a character is a vowel (case-insensitive).	ch	1 if vowel, else 0
removeVowels()	Copies the input string into result without any vowels.	str, result	void
firstRepeatingLowercase()	Finds the leftmost lowercase letter that occurs more than once.	str	The letter, or '\0' if none
areAnagrams()	Compares letter frequencies of two strings, ignoring case and spaces.	s1, s2	1 if anagrams, else 0
main()	Displays the menu and calls the required function.	—	0

7. Sample Output

```
=====
                STRING OPERATIONS MENU
=====
1. Remove all vowels from a string
2. Find first repeating lowercase alphabet
3. Check if two strings are anagrams
4. Exit
-----
Enter your choice (1-4): 1
Enter a string: Programming in C is Fun
String after removing vowels: "Prgrmmng n C s Fn"

Enter your choice (1-4): 2
Enter a string: programming
First repeating lowercase alphabet: 'r'

Enter your choice (1-4): 2
Enter a string: abcdef
No repeating lowercase alphabet found.

Enter your choice (1-4): 3
Enter first string : Listen
Enter second string: Silent
Result: The strings ARE anagrams.

Enter your choice (1-4): 3
Enter first string : Hello
Enter second string: World
Result: The strings are NOT anagrams.

Enter your choice (1-4): 5
Invalid choice! Please enter a number between 1 and 4.

Enter your choice (1-4): 4
Thank you! Exiting the program.
```

8. Test Cases

No.	Operation	Input	Output
1	Remove vowels	Programming in C is Fun	Prgrmmng n C s Fn
2	Remove vowels	AEIOU aeiou	(only the space remains)
3	First repeating	programming	r
4	First repeating	abcdef	No repeating alphabet
5	Anagram	Listen / Silent	Anagrams
6	Anagram	Hello / World	Not anagrams
7	Anagram	Dormitory / Dirty room	Anagrams

9. Conclusion

The program successfully performs all three string operations through a simple menu. Each task is written as a separate function, which keeps the code organised, easy to read and easy to reuse. Remove-vowels works in a single pass over the string, while the repeating-alphabet and anagram checks use a 26-element frequency array, making them efficient with a time complexity of $O(n)$. Through this project, the concepts of strings, functions, loops, arrays and menu-driven programming in C were applied and understood.